



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CNN-Based Traffic Sign Recognition System

COURSE ASSIGNMENT

Submitted in the partial fulfilment for the Course of

MLA0408 - DEEP LEARNING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B Tech AI&ML

Submitted by

K.Giridhar (192425160)

G.Sai Sharan (192425152)

N.Rohith(192425115)

D. Abuzar (192425384)

Under the Supervision of

Dr. Sudhagar D

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

1. Problem Statement:

Traffic signs are an important part of road transportation because they provide information, warnings, restrictions, and directions to drivers. For an intelligent transportation system, it is useful to identify these signs automatically from images captured by cameras.

Manual identification is not suitable for an automated system, especially when a large number of images need to be processed. Traffic signs can also look different depending on their distance from the camera, viewing angle, brightness, background, and image quality. These variations make automatic recognition a challenging image-classification problem.

This project focuses on developing a **CNN-Based Traffic Sign Recognition System**. The system accepts a traffic-sign image as input and predicts the category to which the sign belongs. A labeled collection of traffic-sign images is used to train the model. Before training, the images are resized and normalized, and suitable augmentation techniques are applied to improve the model's ability to recognize signs under different conditions.

A Convolutional Neural Network is used because it can automatically learn visual patterns such as edges, shapes, textures, and important features from images. The trained network is evaluated using test images that were not used during training. Its performance is studied using classification measures and visual results to determine how effectively it recognizes different traffic signs.

2. Objective:

The main objective of this project is to develop an image-based traffic-sign classification system using a **Convolutional Neural Network (CNN)**.

The objectives of the project are:

1. To prepare a labeled dataset containing different categories of traffic signs.
2. To preprocess the input images into a suitable format for CNN training.
3. To use image augmentation to introduce variations in the training images.
4. To design a CNN capable of extracting useful features from traffic-sign images.
5. To train the CNN model using the prepared dataset.
6. To validate the model during the training process and reduce overfitting.
7. To evaluate the trained model using previously unseen test images.
8. To calculate performance measures such as accuracy, precision, recall, and F1-score.
9. To analyze the confusion matrix and identify classes that are frequently misclassified.
10. To display sample predictions and study the effectiveness of the developed system.

3. Requirements and Environment Used:

3.1 Hardware Requirements

The system can be developed and tested using a normal computer with the following basic configuration:

- **Processor:** Intel Core i5 or equivalent
- **RAM:** 8 GB or above
- **Storage:** At least 10 GB of free space
- **GPU:** NVIDIA GPU or cloud GPU, if available
- **Display:** Standard monitor
- **Internet Connection:** Required for downloading the dataset and installing required libraries

A GPU can reduce the time required for CNN training, while the system can also be executed using a CPU for smaller experiments.

3.2 Software Requirements:

The following software and libraries are used for developing the system:

Software / Library	Purpose
Python	Main programming language
TensorFlow / Keras	Building and training the CNN
NumPy	Numerical and array operations
Pandas	Handling tabular information
OpenCV	Image processing
Matplotlib	Creating graphs and visualizations
Scikit-learn	Evaluation metrics and confusion matrix
Google Colab / Jupyter Notebook	Development and execution environment

3.3 Dataset Requirements:

A labeled traffic-sign image dataset is required for training and evaluating the model. Each image should have an associated class label representing the type of traffic sign.

The dataset is divided into separate portions for:

- **Training:** Used to teach the CNN.
- **Validation:** Used to monitor the model during training.
- **Testing:** Used to measure performance on images not used during training.

Keeping the test images separate helps provide a more reliable measurement of the model's classification ability.

3.4 Main Functional Requirements:

The developed system should be able to:

1. Load the traffic-sign images and their labels.
2. Resize images to a common input dimension.
3. Normalize image pixel values.
4. Apply suitable augmentation to training images.
5. Build and train a CNN model.
6. Validate the model during training.
7. Classify unseen traffic-sign images.
8. Display the predicted traffic-sign class.
9. Calculate classification performance.
10. Generate graphs and a confusion matrix for analysis.

3.5 Development Environment:

Programming Language: Python

Framework: TensorFlow/Keras

Development Platform: Google Colab or Jupyter Notebook

Input: Traffic-sign images

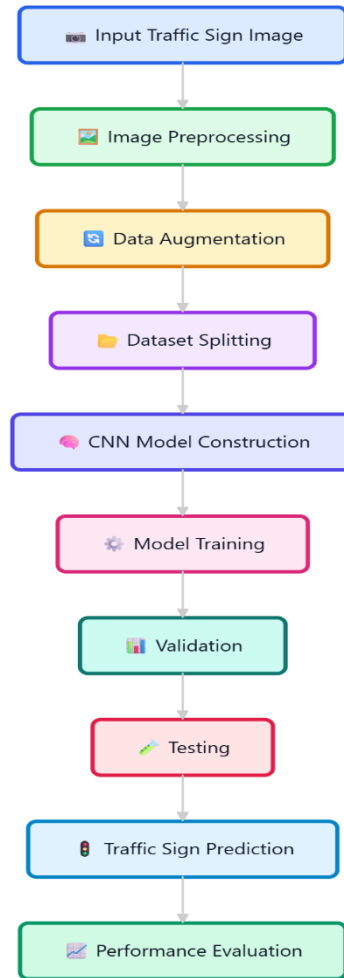
Output: Predicted traffic-sign class and classification performance

The chosen environment provides the libraries needed for image processing, CNN development, model training and result visualization

4. Design / Proposed Solution:

The proposed system uses a **Convolutional Neural Network (CNN)** to recognize traffic signs from input images. The complete process is divided into several stages so that the image can be converted into a suitable form, processed by the trained model, and finally classified.

4.1 System Workflow:



4.2 Input Image:

The system receives an image containing a traffic sign. Images may contain differences in size, brightness, orientation and background. Therefore, the images are converted into a common format before being given to the CNN.

4.3 Image Preprocessing:

The input images are processed using the following steps:

- Resize all images to the same dimensions.
- Convert the images into a suitable numerical representation.
- Normalize pixel values to a smaller range.
- Associate every image with its corresponding class label.

This makes the input data consistent and easier for the CNN to process.

4.4 Data Augmentation:

Training images can be modified slightly to create additional variations. Operations such as small rotations, shifts, zooming and changes in image orientation can be used where suitable.

This helps the model learn features that are not dependent on one particular position or appearance of a traffic sign.

4.5 CNN Model:

The CNN receives the preprocessed image and gradually extracts useful visual information.

A typical structure used in the proposed system is:

Input Layer → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Flatten → Fully Connected Layer → Output Layer

The convolution layers learn important patterns from the images. Pooling reduces the spatial size of the feature maps while retaining significant information. The fully connected section uses the extracted features to determine the traffic-sign category.

4.6 Training and Validation:

The training images are provided to the CNN along with their class labels. During training, the model calculates the difference between its prediction and the actual class. Backpropagation is then used to adjust the network parameters so that the classification error gradually decreases.

Validation data is used during training to observe how well the model performs on images that are not directly used for updating the model parameters.

4.7 Testing and Prediction:

After training is completed, the model is tested using separate images. For a new traffic-sign image, the same preprocessing steps are applied before passing it to the trained CNN.

The output layer produces values corresponding to the possible traffic-sign classes. The class with the highest predicted probability is selected as the final prediction.

4.8 Performance Evaluation

The final model is evaluated using:

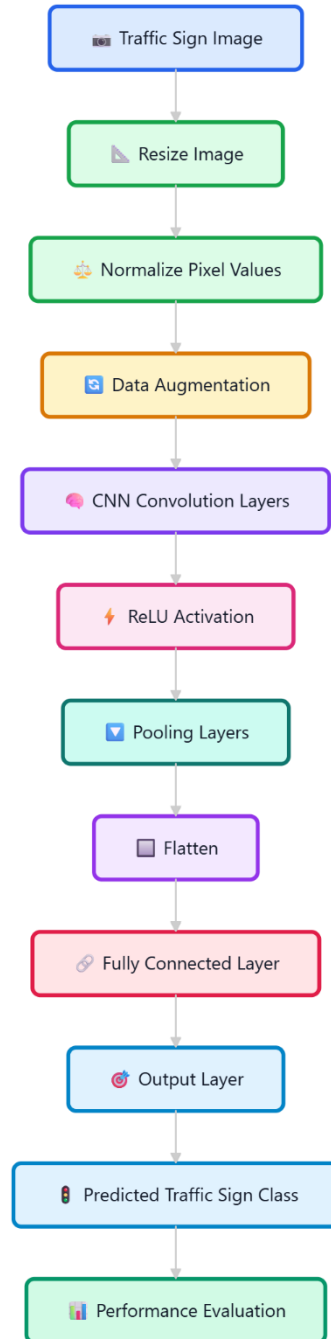
- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

- Training and validation accuracy
- Training and validation loss

Correct and incorrect predictions are also examined to understand the strengths and weaknesses of the developed model.

4.9 Proposed Architecture

The overall design can be represented as:



5. Algorithm / Pseudocode / Flowchart:

5.1 Algorithm

Step 1: Start the system.

Step 2: Load the traffic-sign image dataset along with the corresponding class labels.

Step 3: Resize all images to a fixed input size.

Step 4: Normalize the pixel values so that the images can be processed efficiently by the CNN.

Step 5: Apply suitable augmentation techniques to the training images.

Step 6: Divide the dataset into training, validation, and testing sets.

Step 7: Create the CNN model using convolution, ReLU activation, pooling, flattening, fully connected and output layers.

Step 8: Compile the model using a suitable loss function and optimizer.

Step 9: Train the CNN using the training images.

Step 10: Check the model performance on the validation data after each training cycle.

Step 11: Test the trained model using unseen test images.

Step 12: Calculate accuracy, precision, recall and F1-score.

Step 13: Generate a confusion matrix and performance graphs.

Step 14: Give the predicted traffic-sign class for an input image.

Step 15: Display the final results.

Step 16: Stop.

5.2 Pseudocode

BEGIN

 Load traffic sign dataset

 FOR each image in dataset

 Resize image

 Normalize pixel values

 Assign corresponding class label

 END FOR

 Apply augmentation to training images

 Split dataset into:

 Training data

 Validation data

 Testing data

Create CNN model

Add convolution layers

Add ReLU activation

Add pooling layers

Add flatten layer

Add fully connected layer

Add output layer

Compile CNN model

FOR each training epoch

Train model using training data

Calculate training loss and accuracy

Evaluate model using validation data

Store validation results

END FOR

Load trained model

Test model using unseen test images

Calculate:

Accuracy

Precision

Recall

F1-score

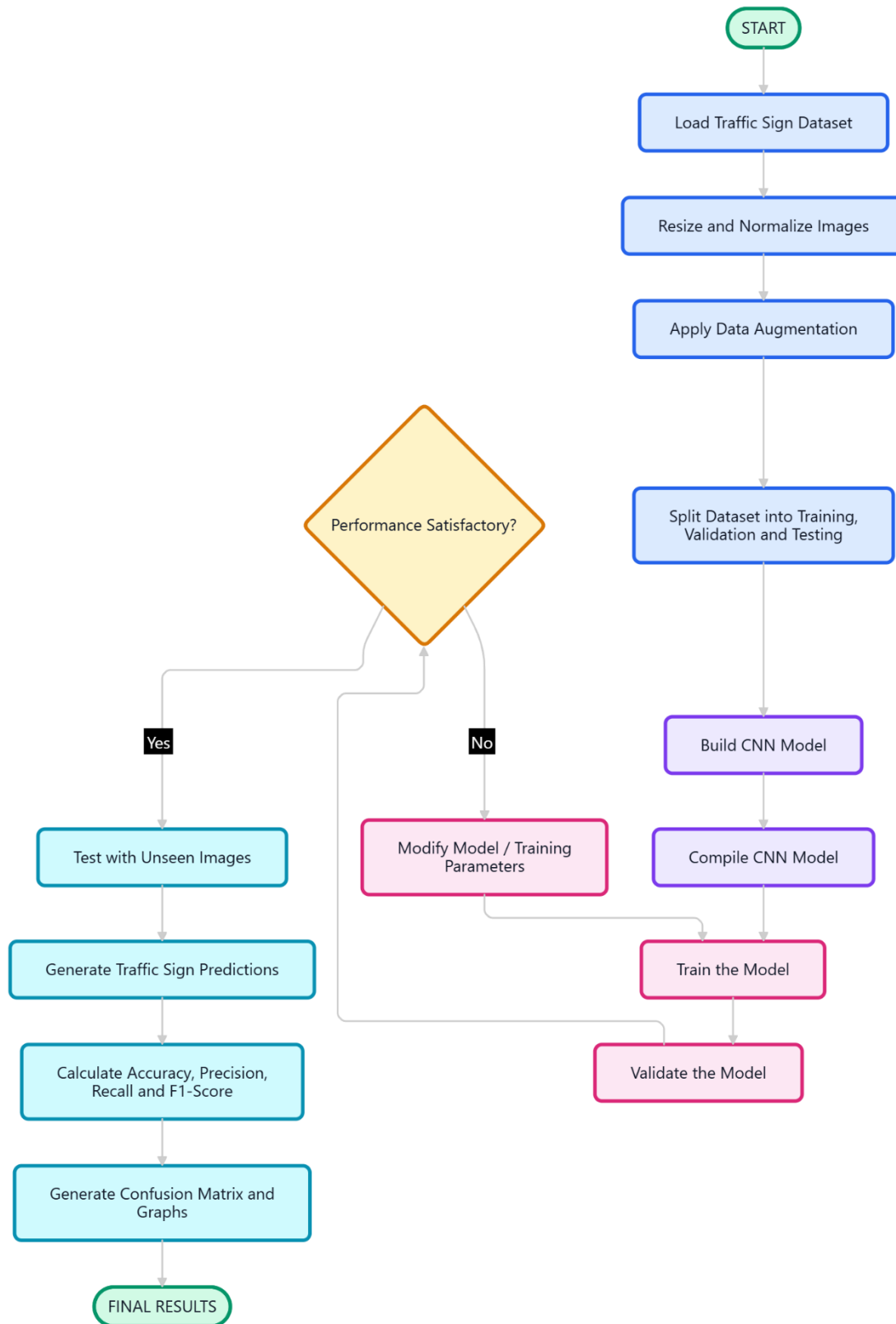
Generate confusion matrix

Display sample predictions

Display performance graphs

END

5.3 Flowchart:



6. Implementation / Source Code:

The proposed traffic sign recognition system was implemented using **Python** in the **Google Colab** environment. TensorFlow and Keras were used to develop and train the Convolutional Neural Network. The implementation includes image preprocessing, data augmentation, CNN model construction, model training, validation, testing, and performance evaluation. The trained model is also used to predict the class of traffic-sign images and display the classification results.

6.1 Implementation Environment:

The system was developed using **Python 3** in **Google Colab**. TensorFlow with Keras was used for constructing and training the CNN model. NumPy was used for numerical operations, while Matplotlib and Seaborn were used to visualize the training results and confusion matrix. Scikit-learn was used to calculate the classification performance metrics.

Component	Tool / Technology
Programming Language	Python
Development Environment	Google Colab
Deep Learning Framework	TensorFlow / Keras
Numerical Processing	NumPy
Data Handling	Pandas
Image Processing	OpenCV
Visualization	Matplotlib, Seaborn
Performance Evaluation	Scikit-learn
Dataset	GTSRB
CNN Input Size	32×32 pixels
Number of Classes	10

6.2 CNN Model Implementation:

The CNN model consists of convolutional layers followed by max-pooling layers for feature extraction. A flattening layer converts the extracted feature maps into a one-dimensional vector. This is followed by a fully connected layer and a dropout layer. The final softmax layer produces the predicted probability for each traffic-sign class.

The model uses the **Adam optimizer** and **categorical cross-entropy loss** for multi-class classification.

6.3 Source Code:

The following code implements the main CNN model used for traffic-sign classification. It defines the convolutional layers, pooling layers, fully connected layer, dropout layer, and output layer.

```
import os
import zipfile
import random
import shutil
```

```

import urllib.request
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import seaborn as sns

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score

IMG_SIZE = 32
BATCH_SIZE = 32
EPOCHS = 10
IMAGES_PER_CLASS = 500
NUM_CLASSES = 10

random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

zip_path = "/content/gtsrb.zip"
extract_path = "/content/gtsrb_raw"

url = "https://sid.erda.dk/public/archives/daaeac0d7ce1152aea9b61d9f1e19370/GTSRB_Final_Training_Images.zip"

urllib.request.urlretrieve(url, zip_path)

if os.path.exists(extract_path):
    shutil.rmtree(extract_path)

os.makedirs(extract_path, exist_ok=True)

with zipfile.ZipFile(zip_path, "r") as z:
    z.extractall(extract_path)

training_root = None

for root, dirs, files in os.walk(extract_path):
    classes = [d for d in dirs if d.isdigit() and len(d) == 5]
    if len(classes) >= 40:
        training_root = root
        break

```

```

if training_root is None:
    raise FileNotFoundError("Dataset directory not found.")

all_classes = sorted([
    d for d in os.listdir(training_root)
    if d.isdigit() and len(d) == 5
])

selected_classes = all_classes[:NUM_CLASSES]

data_path = "/content/traffic_sign_small"

if os.path.exists(data_path):
    shutil.rmtree(data_path)

train_path = os.path.join(data_path, "train")
test_path = os.path.join(data_path, "test")

os.makedirs(train_path, exist_ok=True)
os.makedirs(test_path, exist_ok=True)

for class_id in selected_classes:
    source = os.path.join(training_root, class_id)

    images = [
        f for f in os.listdir(source)
        if f.lower().endswith((".ppm", ".png", ".jpg", ".jpeg"))
    ]

    random.shuffle(images)
    images = images[:IMAGES_PER_CLASS]

    split = int(0.8 * len(images))

    train_images = images[:split]
    test_images = images[split:]

    train_class = os.path.join(train_path, class_id)
    test_class = os.path.join(test_path, class_id)

    os.makedirs(train_class, exist_ok=True)
    os.makedirs(test_class, exist_ok=True)

    for image in train_images:
        shutil.copy2(

```

```

        os.path.join(source, image),
        os.path.join(train_class, image)
    )

    for image in test_images:
        shutil.copy2(
            os.path.join(source, image),
            os.path.join(test_class, image)
        )

train_datagen = ImageDataGenerator(
    rescale=1 / 255.0,
    validation_split=0.2,
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1
)

test_datagen = ImageDataGenerator(
    rescale=1 / 255.0
)

train_data = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    subset="training",
    shuffle=True
)

validation_data = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    subset="validation",
    shuffle=False
)

test_data = test_datagen.flow_from_directory(
    test_path,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="categorical",

```

```

        shuffle=False
    )

    num_classes = train_data.num_classes

    print("Classes:", num_classes)
    print("Training:", train_data.samples)
    print("Validation:", validation_data.samples)
    print("Testing:", test_data.samples)

    images, labels = next(train_data)

    plt.figure(figsize=(10, 8))

    for i in range(min(9, len(images))):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i])
        plt.axis("off")

    plt.suptitle("Sample Traffic Sign Images")
    plt.tight_layout()
    plt.show()

    model = Sequential([
        Conv2D(32, (3, 3), activation="relu", input_shape=(IMG_SIZE, IMG_SIZE, 3)),
        MaxPooling2D((2, 2)),
        Conv2D(64, (3, 3), activation="relu"),
        MaxPooling2D((2, 2)),
        Conv2D(128, (3, 3), activation="relu"),
        MaxPooling2D((2, 2)),
        Flatten(),
        Dense(128, activation="relu"),
        Dropout(0.5),
        Dense(num_classes, activation="softmax")
    ])

    model.compile(
        optimizer="adam",
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )

    model.summary()

    history = model.fit(
        train_data,

```

```
    epochs=EPOCHS,  
    validation_data=validation_data  
)
```

```
plt.figure(figsize=(8, 5))  
plt.plot(history.history["accuracy"], label="Training Accuracy")  
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.title("Training and Validation Accuracy")  
plt.legend()  
plt.grid(True)  
plt.show()
```

```
plt.figure(figsize=(8, 5))  
plt.plot(history.history["loss"], label="Training Loss")  
plt.plot(history.history["val_loss"], label="Validation Loss")  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.title("Training and Validation Loss")  
plt.legend()  
plt.grid(True)  
plt.show()
```

```
test_loss, test_accuracy = model.evaluate(test_data, verbose=1)
```

```
test_data.reset()
```

```
predictions = model.predict(test_data)  
predicted_classes = np.argmax(predictions, axis=1)  
actual_classes = test_data.classes
```

```
class_names = list(train_data.class_indices.keys())
```

```
accuracy = accuracy_score(actual_classes, predicted_classes)  
precision = precision_score(actual_classes, predicted_classes, average="macro", zero_division=0)  
recall = recall_score(actual_classes, predicted_classes, average="macro", zero_division=0)  
f1 = f1_score(actual_classes, predicted_classes, average="macro", zero_division=0)
```

```
print("\nTest Accuracy:", round(accuracy * 100, 2), "%")  
print("Precision:", round(precision * 100, 2), "%")  
print("Recall:", round(recall * 100, 2), "%")  
print("F1-Score:", round(f1 * 100, 2), "%")
```

```
print("\nClassification Report\n")  
print(classification_report(
```



```

    actual_classes,
    predicted_classes,
    target_names=class_names,
    zero_division=0
))

cm = confusion_matrix(actual_classes, predicted_classes)

plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()

test_data.reset()

sample_images, sample_labels = next(test_data)
sample_predictions = model.predict(sample_images, verbose=0)

sample_classes = np.argmax(sample_predictions, axis=1)
sample_confidence = np.max(sample_predictions, axis=1) * 100

plt.figure(figsize=(12, 10))

for i in range(min(12, len(sample_images))):
    plt.subplot(3, 4, i + 1)
    plt.imshow(sample_images[i])
    plt.axis("off")
    plt.title(
        "Predicted: " + class_names[sample_classes[i]] +
        "\nConfidence: " + str(round(sample_confidence[i], 1)) + "%",
        fontsize=9
    )

plt.suptitle("Traffic Sign Prediction Results")
plt.tight_layout()
plt.show()

```

```

model.save("/content/traffic_sign_cnn_model.keras")

print("\nModel Parameters:", model.count_params())
print("Model saved at: /content/traffic_sign_cnn_model.keras")
print("\nCNN Traffic Sign Recognition Completed.")

```

7. Test Cases and Expected / Actual Results:

The CNN model was tested using traffic-sign images that were kept separate from the images used for model training. Different test cases were considered to check the classification ability of the system under different input conditions.

7.1 Test Case Table

Test Case	Input Condition	Expected Result	Actual Result
TC01	Clear traffic-sign image	Correct traffic-sign class should be predicted	To be obtained
TC02	Traffic-sign image with different orientation	Correct class should be identified	To be obtained
TC03	Traffic-sign image with different scale	Correct class should be predicted	To be obtained
TC04	Traffic-sign image with slight image variation	Model should identify the appropriate class	To be obtained
TC05	Unseen traffic-sign image	Model should predict the most probable class	To be obtained
TC06	Visually similar traffic-sign image	Model should distinguish the appropriate class	To be obtained

7.2 Performance Evaluation:

The trained model is evaluated using the test images. Accuracy, precision, recall and F1-score are calculated to measure the classification performance.

Performance Measure	Obtained Value
Test Accuracy	To be filled
Precision	To be filled
Recall	To be filled
F1-Score	To be filled
Number of Classes	10
Number of Model Parameters	To be filled

8. Execution Screenshots / Output:

8.1 Dataset Loading Output:

```
plt.axis("off")
plt.title(
    "Predicted: " + class_names[sample_classes[i]] +
    "\nConfidence: " + str(round(sample_confidence[i], 1)) + "%",
    fontsize=9
)

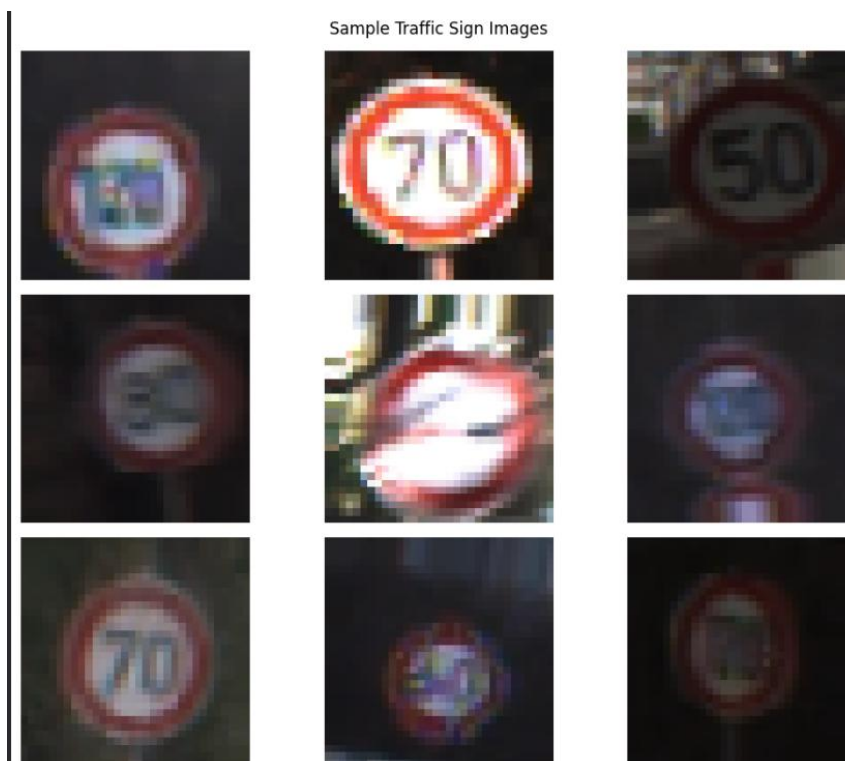
plt.suptitle("Traffic Sign Prediction Results")
plt.tight_layout()
plt.show()

model.save("/content/traffic_sign_cnn_model.keras")

print("\nModel Parameters:", model.count_params())
print("Model saved at: /content/traffic_sign_cnn_model.keras")
print("\nCNN Traffic Sign Recognition Completed.")
```

▼ ... Found 2964 images belonging to 10 classes.
Found 740 images belonging to 10 classes.
Found 926 images belonging to 10 classes.
Classes: 10
Training: 2964
Validation: 740
Testing: 926

8.2 Sample Traffic Sign Images:



8.3 CNN Model Summary:

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.  
super().__init__(activity_regularizer=activity_regularizer, **kwargs)  
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d_6 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_7 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_8 (Conv2D)	(None, 4, 4, 128)	73,856
max_pooling2d_8 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten_2 (Flatten)	(None, 512)	0
dense_4 (Dense)	(None, 128)	65,664
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 10)	1,290

Total params: 160,202 (625.79 KB)

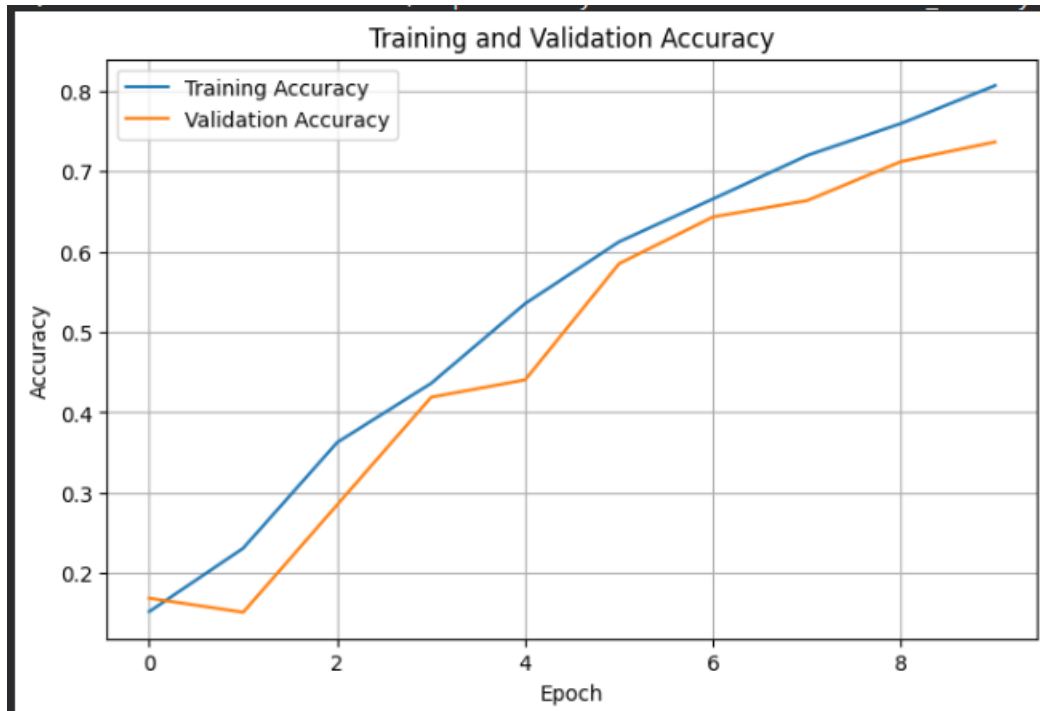
Trainable params: 160,202 (625.79 KB)

Non-trainable params: 0 (0.00 B)

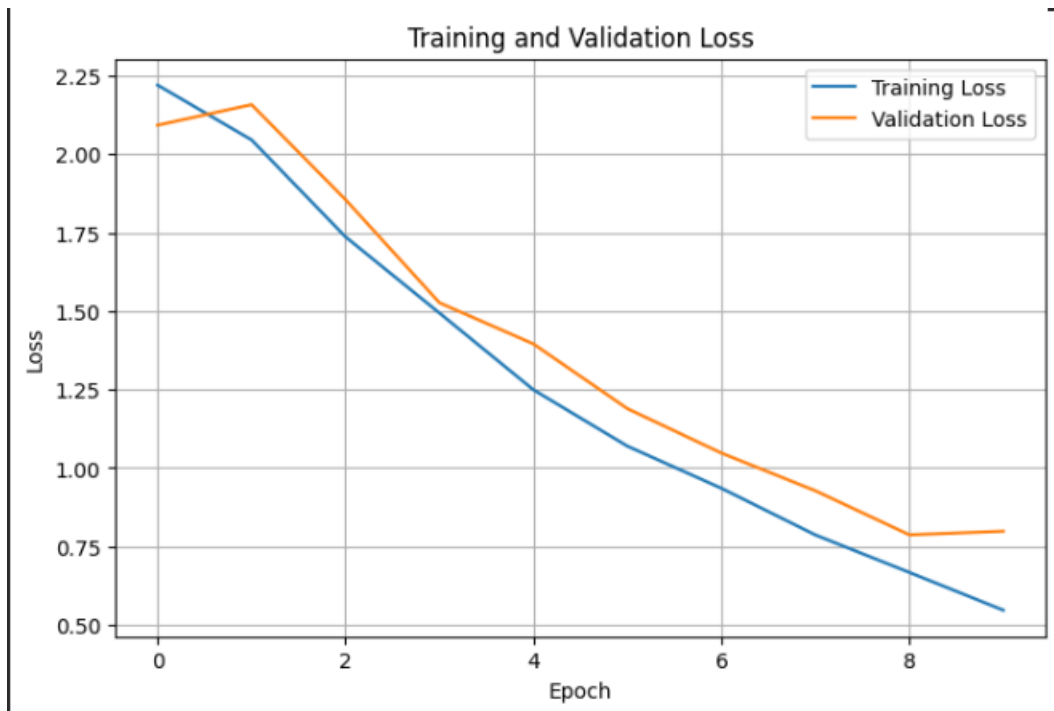
8.4 Training Output:

```
Epoch 1/10  
93/93 ————— 10s 97ms/step - accuracy: 0.1522 - loss: 2.2204 - val_accuracy: 0.1689 - val_loss: 2.0924  
Epoch 2/10  
93/93 ————— 7s 74ms/step - accuracy: 0.2311 - loss: 2.0457 - val_accuracy: 0.1514 - val_loss: 2.1581  
Epoch 3/10  
93/93 ————— 9s 91ms/step - accuracy: 0.3627 - loss: 1.7367 - val_accuracy: 0.2851 - val_loss: 1.8553  
Epoch 4/10  
93/93 ————— 7s 75ms/step - accuracy: 0.4362 - loss: 1.4937 - val_accuracy: 0.4189 - val_loss: 1.5265  
Epoch 5/10  
93/93 ————— 9s 91ms/step - accuracy: 0.5354 - loss: 1.2491 - val_accuracy: 0.4405 - val_loss: 1.3953  
Epoch 6/10  
93/93 ————— 10s 106ms/step - accuracy: 0.6123 - loss: 1.0695 - val_accuracy: 0.5851 - val_loss: 1.1890  
Epoch 7/10  
93/93 ————— 7s 74ms/step - accuracy: 0.6657 - loss: 0.9353 - val_accuracy: 0.6432 - val_loss: 1.0475  
Epoch 8/10  
93/93 ————— 9s 92ms/step - accuracy: 0.7196 - loss: 0.7865 - val_accuracy: 0.6635 - val_loss: 0.9273  
Epoch 9/10  
93/93 ————— 7s 74ms/step - accuracy: 0.7594 - loss: 0.6670 - val_accuracy: 0.7122 - val_loss: 0.7869  
Epoch 10/10  
93/93 ————— 8s 91ms/step - accuracy: 0.8067 - loss: 0.5466 - val_accuracy: 0.7365 - val_loss: 0.7984
```

8.5 Training and Validation Accuracy:



8.6 Training and Validation Loss:



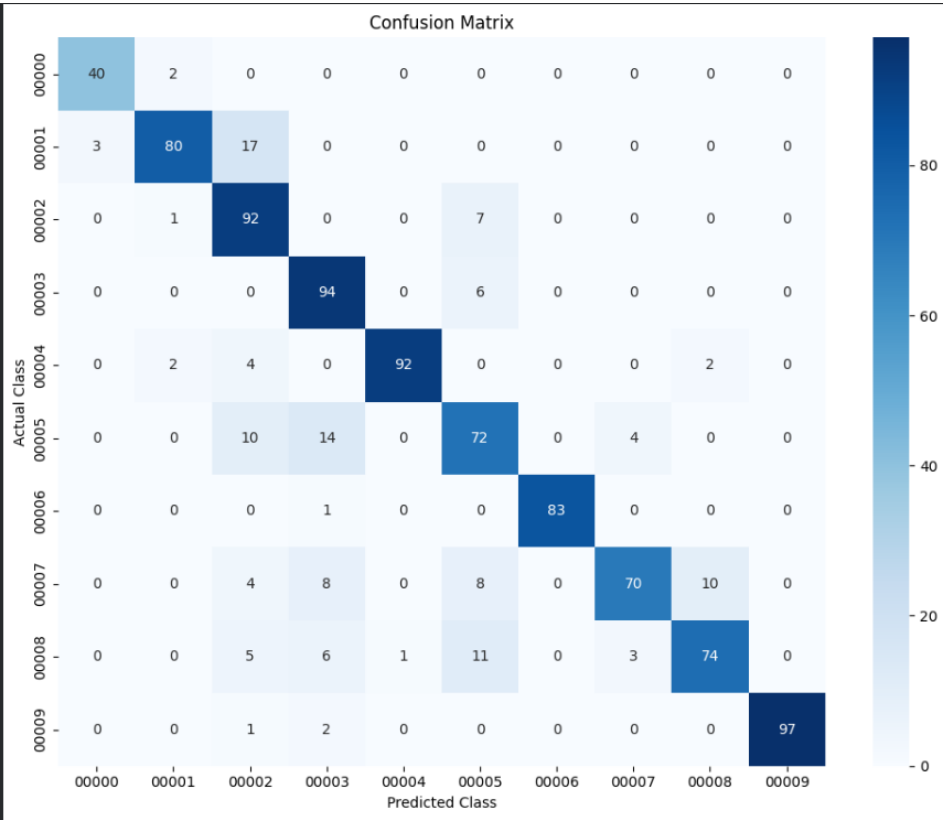
8.7 Classification Performance:

Test Accuracy: 85.75 %
Precision: 87.66 %
Recall: 86.5 %
F1-Score: 86.56 %

Classification Report

	precision	recall	f1-score	support
00000	0.93	0.95	0.94	42
00001	0.94	0.80	0.86	100
00002	0.69	0.92	0.79	100
00003	0.75	0.94	0.84	100
00004	0.99	0.92	0.95	100
00005	0.69	0.72	0.71	100
00006	1.00	0.99	0.99	84
00007	0.91	0.70	0.79	100
00008	0.86	0.74	0.80	100
00009	1.00	0.97	0.98	100
accuracy			0.86	926
macro avg	0.88	0.87	0.87	926
weighted avg	0.87	0.86	0.86	926

8.8 Confusion Matrix:



8.9 Prediction Results:



9. Analysis and Discussion

9.1 Model Performance Analysis:

The developed CNN model was evaluated using the test images prepared from the selected traffic-sign classes. The performance was measured using accuracy, precision, recall and F1-score. These metrics provide a broader understanding of the classification capability of the proposed model.

Accuracy represents the overall percentage of correctly classified traffic-sign images. Precision indicates how accurately the model identifies a particular class when it makes a prediction for that class. Recall measures the ability of the model to correctly detect images belonging to the actual class. The F1-score combines precision and recall and provides a balanced measure of classification performance.

The results indicate how effectively the compact CNN can learn visual features from traffic-sign images and distinguish between the selected categories.

9.2 Training and Validation Analysis:

The training and validation accuracy graph shows how the CNN performance changes over the training epochs. In general, the training accuracy is expected to improve as the network learns useful visual features from the images. The validation accuracy provides an indication of how well the learned features generalize to images that were not used directly for model parameter updates.

The training and validation loss curves provide another view of the learning process. A decreasing training loss indicates that the model is progressively reducing its classification error. If the validation loss also decreases and remains reasonably close to the training loss, the model demonstrates good generalization. A large and continuous difference between training and validation performance may indicate overfitting.

The graphs generated during execution are therefore used to determine whether the selected CNN architecture provides stable learning behaviour.

9.3 Confusion Matrix Analysis

The confusion matrix provides a class-wise view of the prediction results. The diagonal elements represent correctly classified images, while the off-diagonal elements indicate misclassification between different traffic-sign categories.

A strong diagonal pattern indicates that the model is successfully distinguishing most of the selected traffic-sign classes. Misclassifications may occur when two signs have similar shapes, colours or visual structures. The confusion matrix is particularly useful because overall accuracy alone cannot show which specific classes are responsible for classification errors.

The confusion matrix generated from the test dataset is included in Section 8 and is used to identify the classes that require further improvement.

9.4 Prediction Analysis

The trained CNN was also used to predict individual traffic-sign images. For every input image, the softmax output provides probabilities for the selected classes. The class having the highest probability is considered the predicted class.

The prediction results include the predicted class and its confidence value. Correct predictions with high confidence indicate that the model has learned distinctive visual characteristics for those signs. Lower-confidence predictions may indicate images that are visually unclear, differently oriented, poorly illuminated or similar to another traffic-sign category.

9.5 Advantages of the Proposed System

The proposed system has several practical advantages. The CNN automatically learns relevant image features instead of requiring manually designed feature-extraction rules. Data augmentation

introduces small variations in rotation, position and scale, helping the model handle moderate changes in image appearance.

9.6 Limitations

The experiment uses a controlled subset of the GTSRB dataset containing ten selected classes and a limited number of images per class. Therefore, the results should not be interpreted as the performance of a complete traffic-sign recognition system covering every available category.

The input images are resized to 32×32 pixels, which reduces computational requirements but can remove some fine visual details. Performance may also be affected by difficult conditions such as severe illumination changes, image blur, occlusion, unusual viewing angles and visually similar signs.

9.7 Possible Improvements:

The system can be improved by training with all available traffic-sign classes and a larger number of images. More advanced CNN architectures or transfer-learning models could also be investigated to improve feature extraction and classification performance.

Further experiments can include brightness variation, stronger augmentation, image-quality degradation and partial occlusion to evaluate the robustness of the model. Explainable-AI methods such as Grad-CAM could also be used to understand which regions of a traffic-sign image influence the CNN prediction.

Overall, the experiment demonstrates the complete deep-learning workflow from image preparation and CNN training to quantitative evaluation and visual prediction analysis.

10. Conclusion:

The CNN-based Traffic Sign Recognition System was successfully designed to demonstrate an image classification approach for recognizing selected traffic-sign categories. The system performs the complete workflow of dataset preparation, image resizing, normalization, data augmentation, feature extraction, CNN training, validation and testing.

The proposed CNN uses multiple convolutional and pooling layers to learn important visual features from traffic-sign images. The fully connected layer and softmax output layer then perform the final classification. Performance was evaluated using accuracy, precision, recall, F1-score and a confusion matrix.

The experimental results obtained from the Colab implementation provide an understanding of the model's classification capability and its behaviour across different traffic-sign classes. The prediction outputs also demonstrate how the trained network assigns a class and confidence value to an input image.

Although the present implementation is limited to a selected subset of traffic-sign classes, it establishes a practical foundation for a larger recognition system. Future work can include all available classes, larger datasets, transfer-learning models and testing under challenging real-world conditions.

Overall, the project demonstrates how convolutional neural networks can be applied to traffic-sign image recognition and provides practical experience in developing, training and evaluating a deep-learning classification system.

11. Student Reflection:

1. K.Giridhar — Dataset & Preprocessing

Working on the dataset preparation stage helped me understand that the quality of input images has a direct effect on CNN performance. I learned how to organize traffic-sign images into different classes, resize them to a common dimension and normalize pixel values before training. I also understood the importance of augmentation techniques such as rotation, shifting and zooming for creating useful variations in training images. Given more time, I would experiment with additional preprocessing techniques such as brightness adjustment, blur and contrast variation to study their effect on recognition accuracy.

2. G.Sai Sharan— CNN Architecture & Design

This project helped me understand how a CNN gradually learns visual features from traffic-sign images. I worked on understanding the purpose of convolution, ReLU activation, max-pooling, flattening and fully connected layers in the proposed architecture. Choosing the number of filters and layers showed me that model design involves balancing recognition performance with computational requirements. With additional resources, I would compare this compact CNN with deeper architectures or transfer-learning models to determine whether more complex feature extraction improves classification of visually similar signs.

3. N.Rohith — Implementation & Training

The implementation stage gave me practical experience in converting the proposed design into an executable deep-learning program using Python and TensorFlow/Keras. I learned how to configure the optimizer, loss function, batch size and number of training epochs and how to monitor training and validation performance. Observing the accuracy and loss curves helped me understand how a model learns and how overfitting can be identified. Given more time, I would perform systematic experiments with learning rate, dropout, batch size and training epochs to determine a better combination of hyperparameters.

4. D.Abuzar — Evaluation & Analysis

The evaluation stage taught me that judging a classification model requires more than looking at overall accuracy. I learned how precision, recall, F1-score and the confusion matrix provide additional information about class-wise performance. Examining incorrect predictions helped me understand that visually similar traffic signs can be difficult for the model to distinguish. With more time, I would investigate the misclassified images in greater detail and use techniques such

as Grad-CAM to understand which regions of the traffic-sign image influenced the CNN's prediction.

12. Individual Contribution of Group Members:

S. No	Name	Register No.	Contribution
1	K.Giridhar	192425160	Problem formulation and requirements analysis, dataset selection and preparation, preprocessing and augmentation, documentation of Sections 1–3
2	G.Sai Sharan	192425152	System design, CNN architecture design and justification, algorithm, pseudocode and flowchart, documentation of Sections 4–5
3	N. Rohith	192425115	CNN implementation, model training and hyperparameter configuration, test-case design, documentation of Sections 6–7
4	D. Abuzar	192425384	Model evaluation, performance metrics and visualizations, results analysis, conclusion and report compilation, documentation of Sections 8–13

13. References

1. Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32, 323–332.
2. German Traffic Sign Recognition Benchmark (GTSRB). Traffic Sign Recognition Benchmark Dataset. Ruhr-Universität Bochum.
3. TensorFlow. TensorFlow Documentation: Machine Learning and Deep Learning Framework.
4. Keras. Keras Documentation: Deep Learning API for TensorFlow.
5. Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications.
6. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
7. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.
8. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
9. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
10. Waskom, M. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021.